

A Visual Guide to LLMs (Part 2)

Inside the Transformer Architecture.



ROHIT KUMAR TIWARI

JUN 07, 2026

 16



 3

Share

...

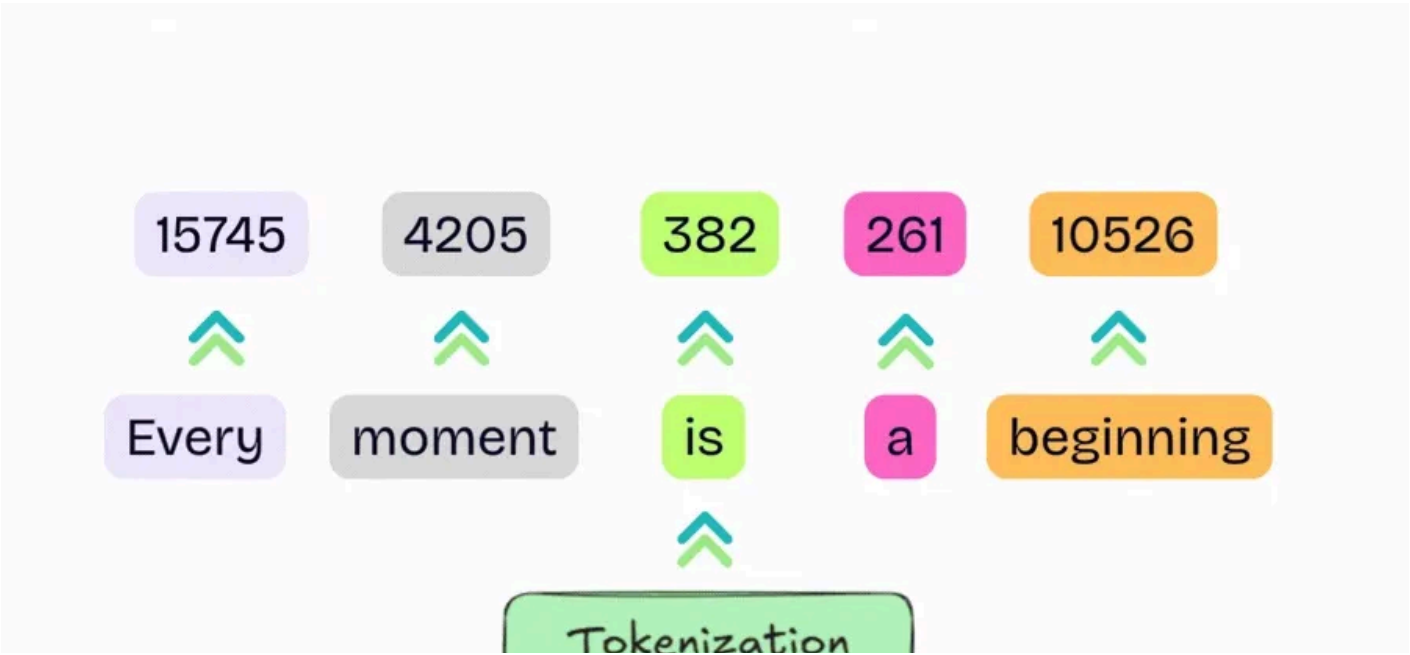
In this two part series, we will go through the core components of Large Language Model (LLM) architecture step by step, making it easy to understand, **even if you are new to AI.**

Series Roadmap

Part 1: This part will show how written language is split into tokens and then turned into input embedding vectors that a model can work with. It explains tokenization, token embeddings, and positional embeddings.

A Visual Guide to LLMs (Part 1)

ROHIT KUMAR TIWARI • JAN 18

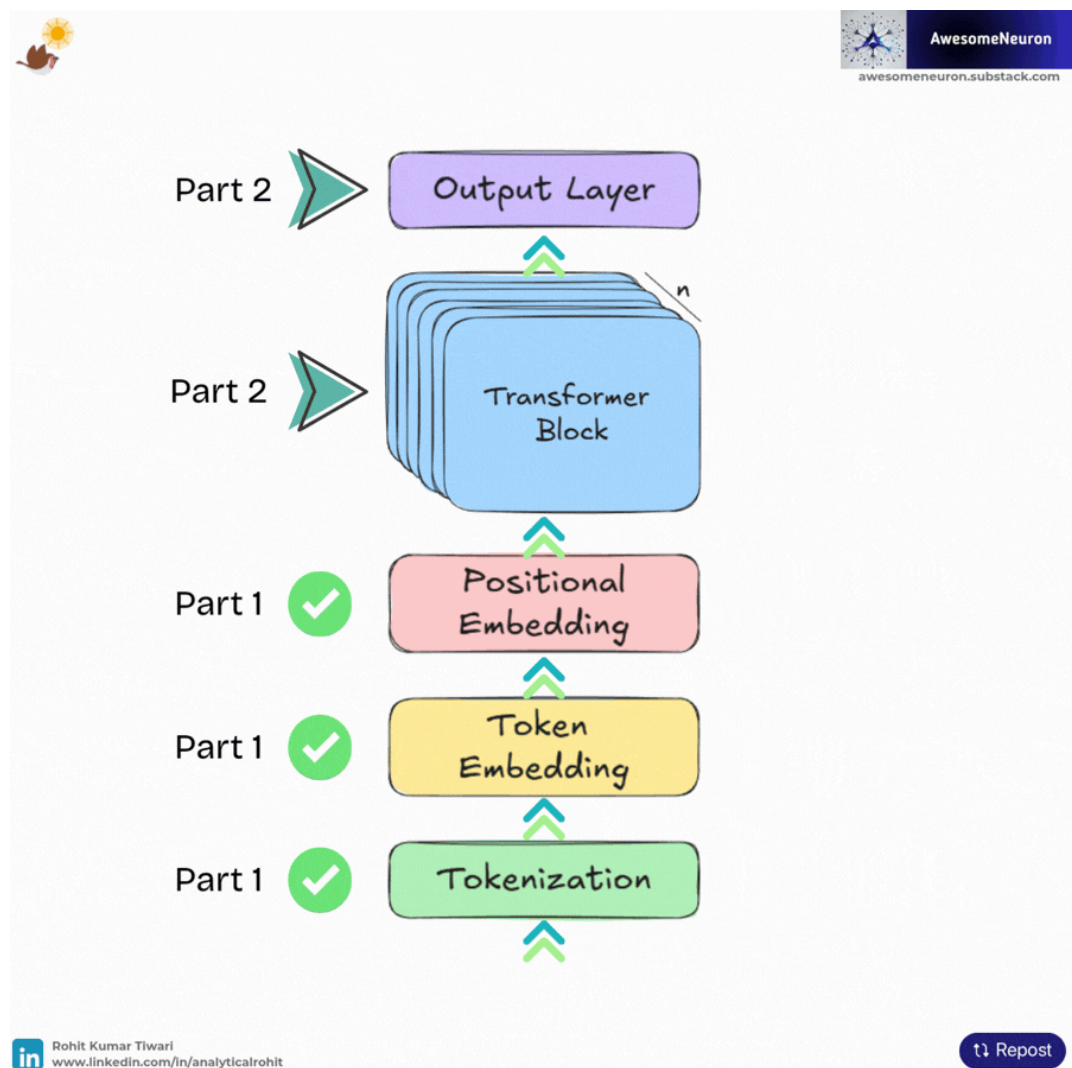


In this two part series, we will go through the core components of Large Language Model (LLM) architecture step by step, making it easy to understand, even if you are new to AI.

[Read full story](#) →

Part 2: This part will show how those input embedding vectors move through the transformer block to produce output. It covers self attention, causal self attention and masked multi-head attention mechanisms, feedforward neural networks, how the next token is chosen, and how training slowly improves the model over time.

In [A Visual Guide to LLMs \(Part 1\)](#), we learned how Large Language Models (LLMs) take human language and turn it into something machines can understand. We took the sentence, “Every moment is a beginning,” sliced it into tokens, assigned them numerical IDs, and converted them into dense vector representations called token embeddings. We also added positional embeddings so the model knows the exact order of the words.



In Part 2, we will explore: the **Transformer Block** and the **Output layer**.

Now, I will explain each component of the Transformer block one by one.

One of the most important components is Masked Multi-Head Attention. However, before we get to that, we will first look at the Self Attention and Causal Self Attention mechanisms. Understanding these concepts will make it much easier to fully grasp how Masked Multi-Head Attention works. We will also cover Feedforward Neural Networks, Residual Connections, and Layer Normalization to understand how all the components of a Transformer block work together.

1. Self Attention Mechanism

Language is highly contextual. If you read the word “beginning,” its exact meaning depends heavily on the words that came before it. The self attention mechanism allows the model to understand how words relate to each other. Instead of reading each word in isolation, every token looks at the other tokens and decides which ones matter most.

$$Attention(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) \cdot V$$

In Part 1, we already created **Input Embeddings** after summing token and position embeddings.

Example: Every moment is a beginning

Token IDs: {”Every”: 15745, “moment”: 4205, “is”: 382, “a”: 261, “beginning”: 10526}

Final Input Embeddings (Sum of Token Embeddings + Positional Embeddings):

- “15745” (position 1): [-0.5880, 0.3486, 0.6603] + [-0.9178, 0.9045, -2.0975]
- “4205” (position 2): [-0.2196, -0.3792, 0.7671] + [1.1558, -1.2157, 0.1295]
- “382” (position 3): [-1.1925, 0.6984, -1.4097] + [1.0937, 0.2066, 3.1815]

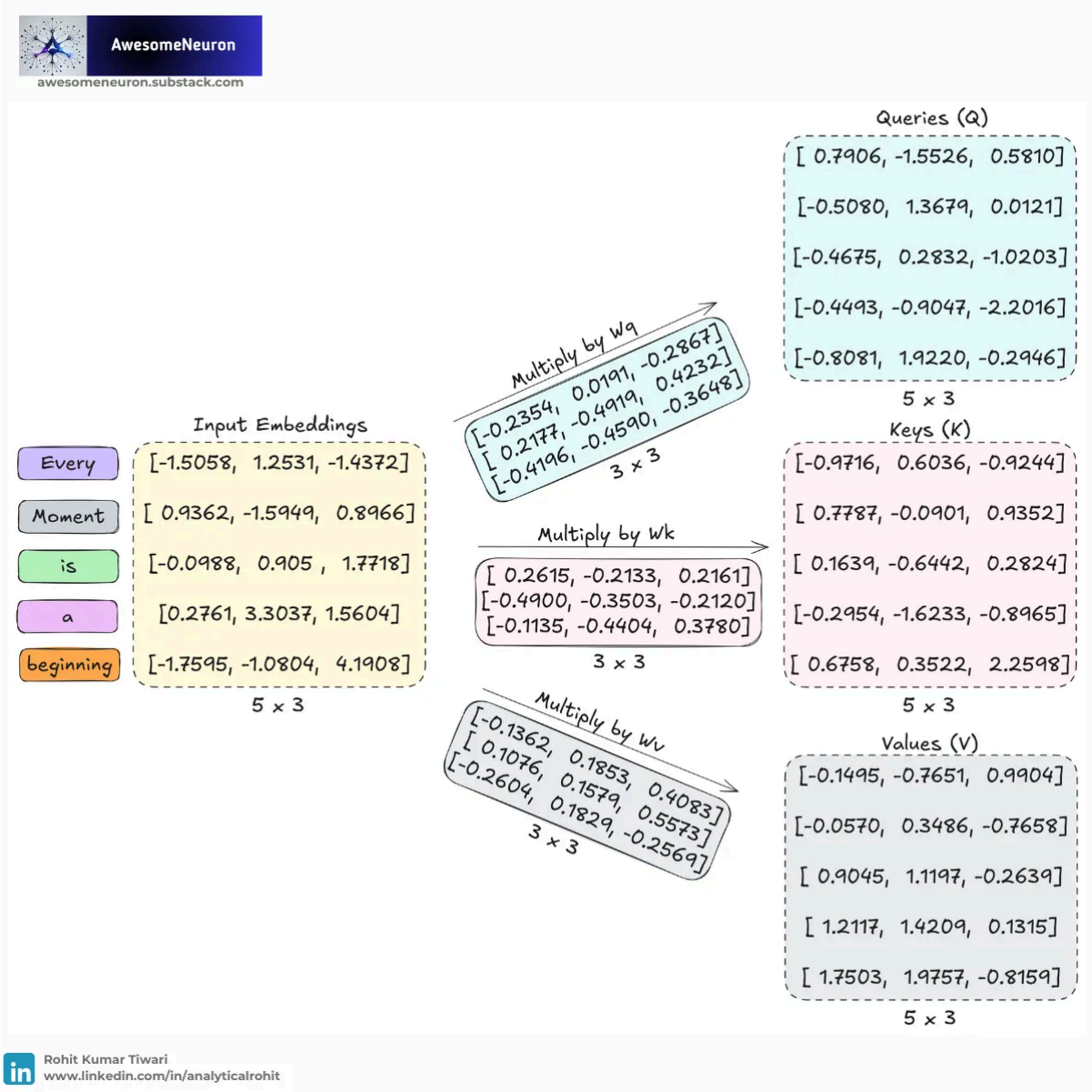
- “261” (position 4): [0.1794, 1.8951, 1.3689] + [0.0967, 1.4086, 0.1915]
- “10526” (position 5): [-1.6033, -1.3250, 0.1784] + [-0.1562, 0.2446, 4.0124]

After performing element wise addition:

- “15745” (position 1): [-1.5058, 1.2531, -1.4372]
- “4205” (position 2): [0.9362, -1.5949, 0.8966]
- “382” (position 3): [-0.0988, 0.905 , 1.7718]
- “261” (position 4): [0.2761, 3.3037, 1.5604]
- “10526” (position 5): [-1.7595, -1.0804, 4.1908]

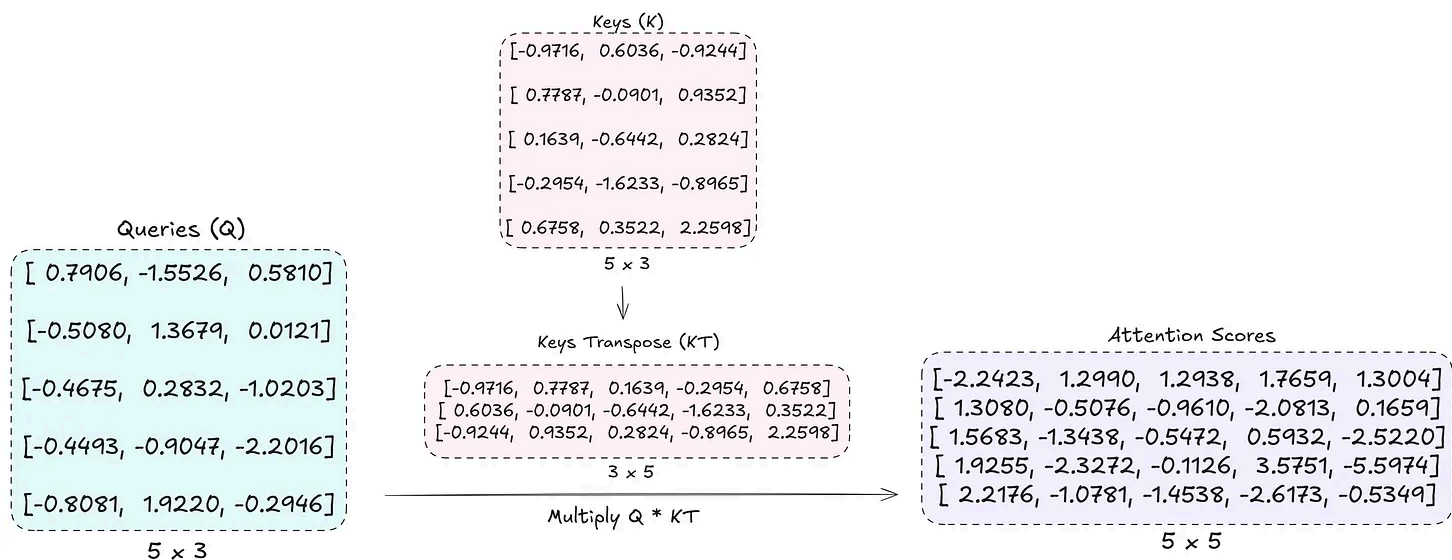
To understand the word “beginning” in our sentence, the model pays attention to words like “moment” and “Every”. This gives context and helps the model capture the idea that each moment can represent a fresh start.

Step 1: To achieve this mathematically, the model uses three vectors derived from the input embeddings: **Queries (Q)**, **Keys (K)**, and **Values (V)**.



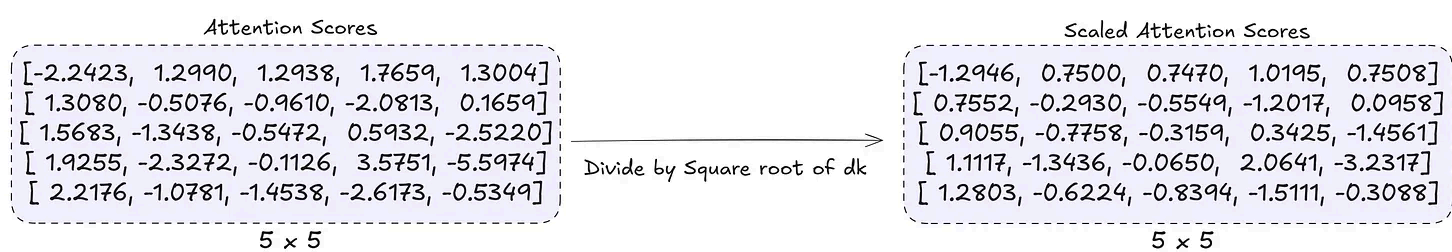
Step 2: We compute the dot product between all Queries and Keys to measure how well they match.

$$\text{Attention Scores} = QK^T$$



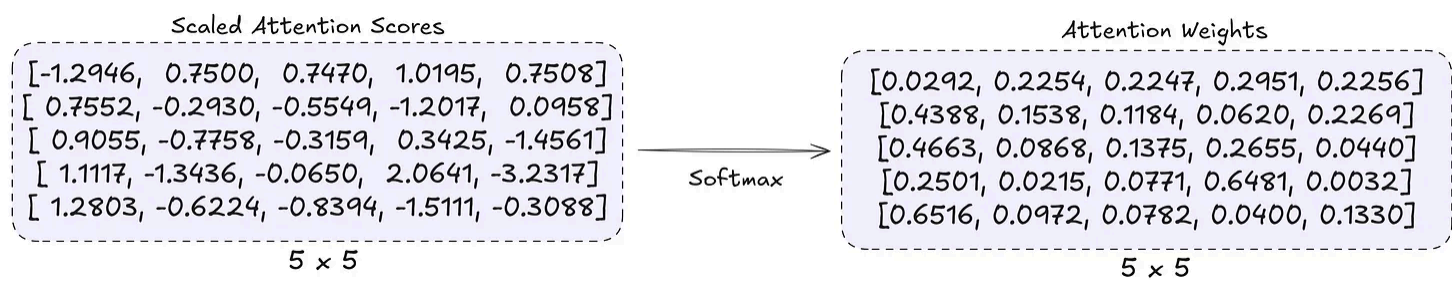
Step 3: The result is scaled by the square root of the key dimension d_k to keep values stable during training.

$$\text{Scaled Attention Scores} = \frac{QK^T}{\sqrt{d_k}}$$



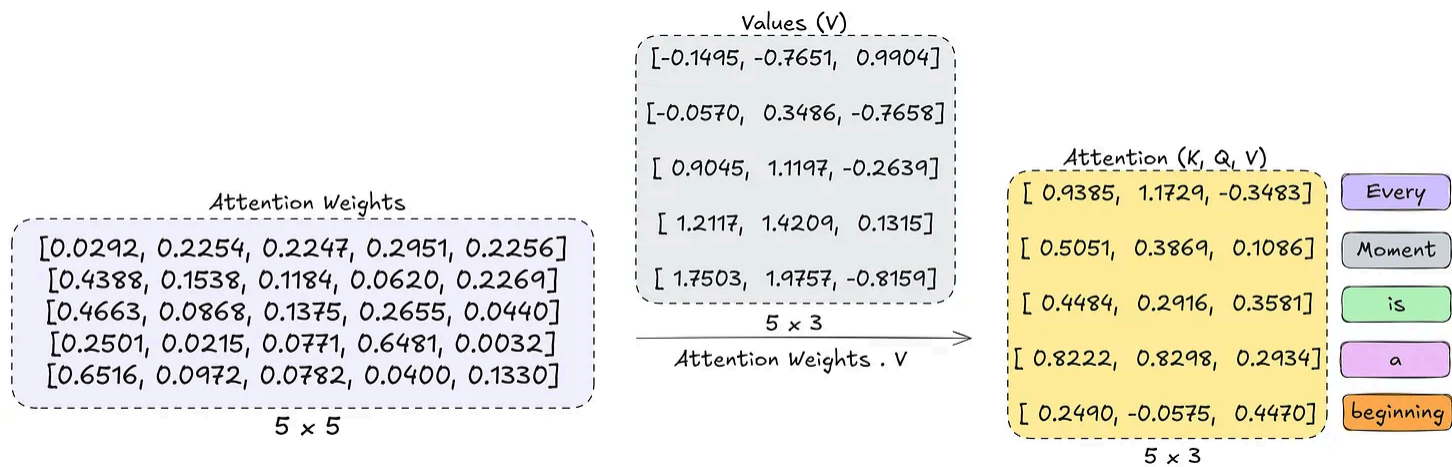
Step 4: Apply softmax to obtain attention weights.

$$\text{Attention Weights} = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right)$$

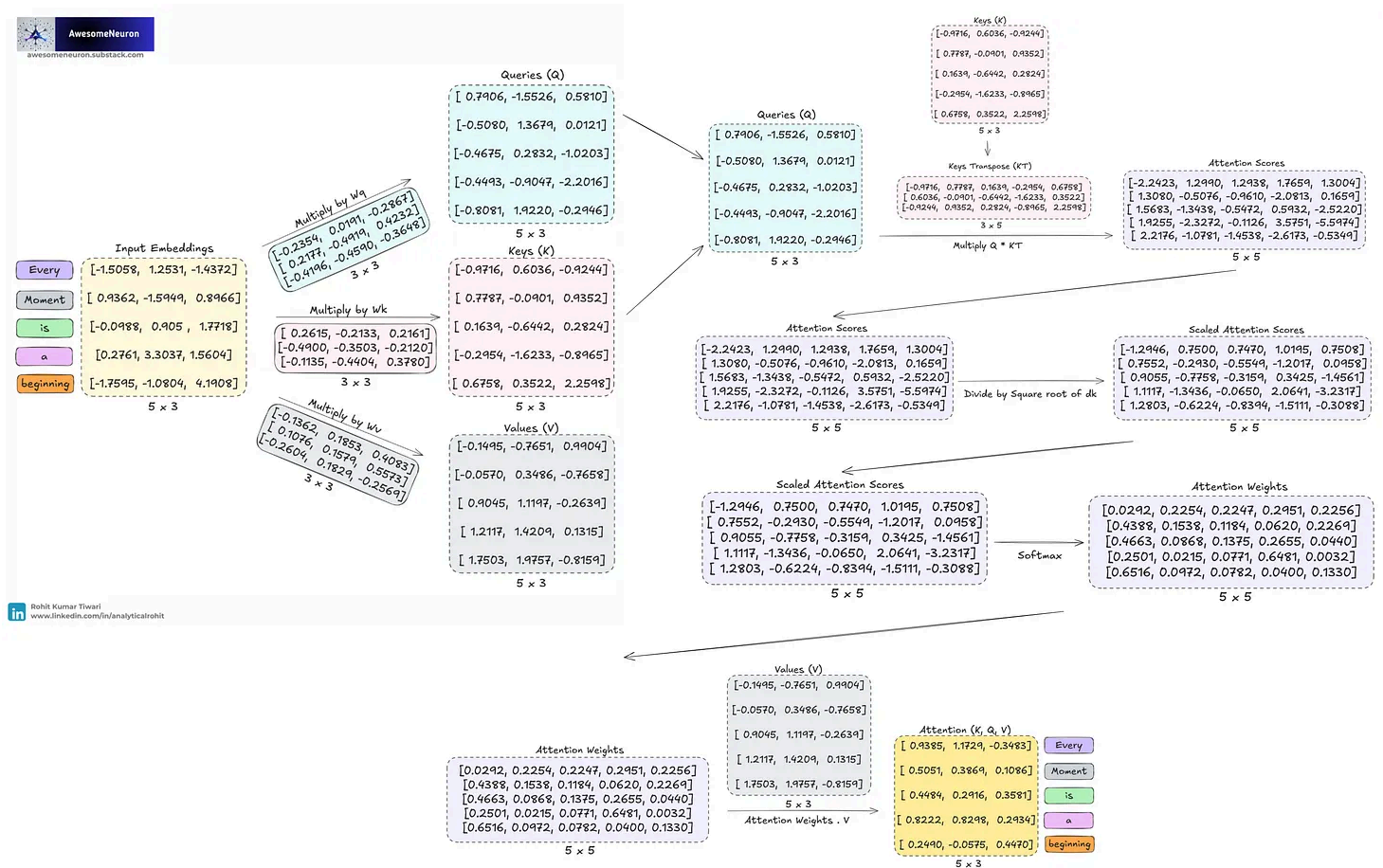


Step 5: Calculate context vectors.

$$\text{Context Vectors} = \text{Attention Weights} \cdot V = \text{Attention}(Q, K, V)$$



Complete Self Attention:



After attention each token now contains information gathered from other tokens in the sequence. This is the core idea behind transformers.

Causal Masking: Hiding the Future Tokens

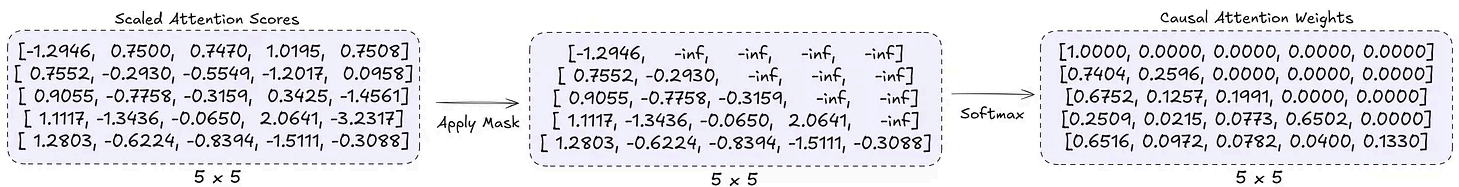
In standard self-attention, each token can attend to all other tokens. But in language models generating text word-by-word, future tokens should not be visible during prediction.

Causal self attention mechanism uses a mask to prevent tokens from attending to future tokens. The allowed attention pattern forms a lower triangular matrix, ensuring that each token can attend only to itself and earlier tokens in the sequence.

$$\begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 & 0 \\ 1 & 1 & 1 & 0 & 0 \\ 1 & 1 & 1 & 1 & 0 \\ 1 & 1 & 1 & 1 & 1 \end{bmatrix}$$

- 1 means attention is allowed.
- 0 means attention is blocked.

This is implemented by masking the blocked positions and replacing their attention scores with negative infinity before applying the softmax function. After softmax, these positions receive a probability of 0, preventing the model from attending to future tokens.



This ensures:

- Token 1 sees only itself.
- Token 2 sees tokens 1 and 2.

- Token 3 sees tokens 1, 2, and 3.

Now each token can only attend to itself and previous tokens.

Here is the code showing causal self attention mechanism in action.

```

1  class CausalSelfAttention(nn.Module):
2      def __init__(self, d_model):
3          super().__init__()
4          self.d_model = d_model
5
6          # Three linear layers that project input -> Q, K, V
7          self.q_proj = nn.Linear(d_model, d_model, bias=False)
8          self.k_proj = nn.Linear(d_model, d_model, bias=False)
9          self.v_proj = nn.Linear(d_model, d_model, bias=False)
10
11     def forward(self, x):
12         Q = self.q_proj(x)
13         K = self.k_proj(x)
14         V = self.v_proj(x)
15
16         # Scaled dot-product attention
17         # For simplicity, this example assumes a single sequence and omits batching.
18         scores = Q @ K.T / (self.d_model ** 0.5)
19
20         # Create causal mask
21         seq_len = x.shape[0]
22         mask = torch.triu(torch.ones(seq_len, seq_len), diagonal=1)
23
24         # Prevent attention to future tokens
25         scores = scores.masked_fill(mask == 1, float('-inf'))
26
27         # Convert scores into probabilities
28         attn_weights = torch.softmax(scores, dim=-1)
29
30         # Compute context vectors
31         context_vec = attn_weights @ V
32
33         return context_vec, attn_weights

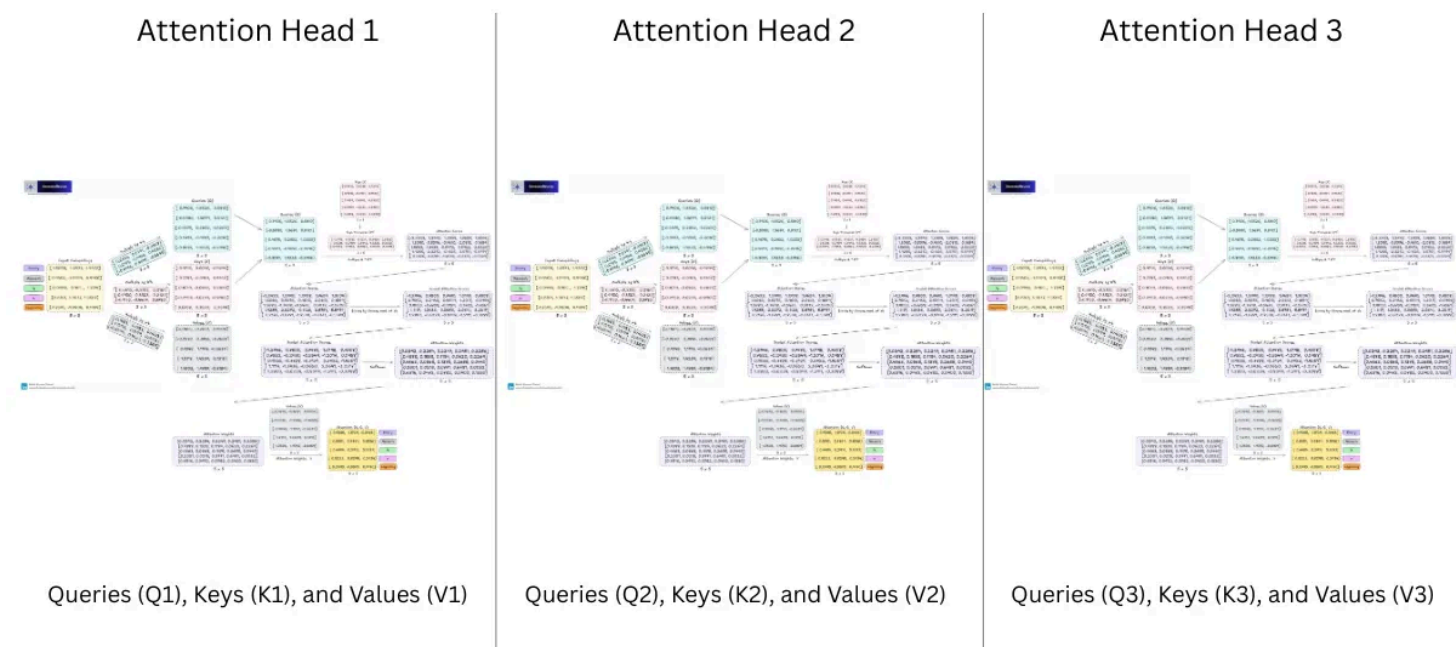
```

2. Masked Multi-Head Attention Mechanism

A single attention head has limited capacity to capture the many different relationships present in language. Multiple attention heads allow the model to learn different patterns simultaneously. To capture grammar, meaning, long-range dependencies, and subject-object relationships simultaneously, the model uses Masked Multi-Head Attention. Multiple attention heads run in parallel, looking at the exact same sentence differently.

For “Every moment is a beginning,” different heads might focus on different nuances:

- **Attention Head 1 (Meaning):** Connects “moment” $\longleftrightarrow$ “beginning” to understand the concept of renewal.
- **Attention Head 2 (Grammar):** Connects “Every” $\longleftrightarrow$ “moment” to understand that “Every” is describing “moment”.
- **Attention Head 3 (Structure):** Connects “is” $\longleftrightarrow$ “beginning” to anchor the main statement of the sentence.



How it works:

1. The input embeddings are split into smaller parts called heads.
2. Each head performs attention independently on its own dimensions.
3. The outputs from all heads are concatenated together.
4. A final linear layer combines this diverse information into one unified, context-rich representation.

Here is the code showing Masked Multi-Head Attention in action.

```
1  class MaskedMultiHeadAttention(nn.Module):
2
3      def __init__(self, d_model, num_heads):
4          super().__init__()
5
6          assert d_model % num_heads == 0, \
7              "d_model must be divisible by num_heads"
8
9          self.d_model = d_model
10         self.num_heads = num_heads
11         self.head_dim = d_model // num_heads
12
13         # QKV projections
14         self.q_proj = nn.Linear(d_model, d_model, bias=False)
15         self.k_proj = nn.Linear(d_model, d_model, bias=False)
16         self.v_proj = nn.Linear(d_model, d_model, bias=False)
17
18         # Final projection
19         self.out_proj = nn.Linear(d_model, d_model, bias=False)
20
21     def forward(self, x):
22
23         b, seq_len, _ = x.shape
24
25         # Project to Q, K, V
26         Q = self.q_proj(x)
27         K = self.k_proj(x)
28         V = self.v_proj(x)
29
30         # Split into heads
31         Q = Q.view(b, seq_len, self.num_heads, self.head_dim)
32         K = K.view(b, seq_len, self.num_heads, self.head_dim)
33         V = V.view(b, seq_len, self.num_heads, self.head_dim)
34
35         # Move heads before seq_len (b, num_heads, seq_len, head_dim)
36         Q = Q.transpose(1, 2)
37         K = K.transpose(1, 2)
38         V = V.transpose(1, 2)
39
```



```

40         # Attention scores
41         scores = Q @ K.transpose(-2, -1)
42         scores = scores / (self.head_dim ** 0.5)
43
44         # Causal mask
45         mask = torch.triu(
46             torch.ones(seq_len, seq_len, device=x.device),
47             diagonal=1
48         ).bool()
49
50         scores = scores.masked_fill(mask, float('-inf'))
51
52         # Attention probabilities
53         attn_weights = torch.softmax(scores, dim=-1)
54
55         # Context vectors
56         context = attn_weights @ V
57
58         # Combine heads
59         # (b, num_heads, seq_len, head_dim)
60         # ->
61         # (b, seq_len, num_heads, head_dim)
62
63         context = context.transpose(1, 2).contiguous()
64
65         # Merge heads
66         context = context.view(b, seq_len, self.d_model)
67
68         # Final projection
69         output = self.out_proj(context)
70
71         return output

```

3. Feedforward Neural Networks (FFN)

Attention allows tokens to communicate and mix information across the sequence. But after this information is gathered, each token still needs individual processing to learn complex patterns. This is the role of the Feedforward Network (often called the FFN or MLP block).

A feedforward neural network typically consists of two linear layers with an activation function (like GELU) in between, temporarily expanding the hidden dimension to help the model learn more complex patterns.

1. **Linear layer:** Temporarily expands the hidden dimension (often by 4x) to give the model space to learn highly complex patterns.
2. **Activation function:** Introduces non-linearity so the model can understand complex, non-straightforward relationships.
3. **Second linear layer:** Compresses the dimension back to its original size.

Here is the code showing FFN in action.

```

1     class FeedForward(nn.Module):
2         def __init__(self, d_model, hidden_dim):
3             super().__init__()
4             self.net = nn.Sequential(
5                 nn.Linear(d_model, hidden_dim), # Linear layer
6                 nn.GELU(), # Activation function Modern GPTs use GELU
7                 nn.Linear(hidden_dim, d_model), # Second linear layer
8             )
9
10        def forward(self, x):
11            return self.net(x)

```


4. Residual Connections

As networks become deeper, gradients (the signals used to teach the model) can become extremely small or large during backpropagation, known as the vanishing or exploding gradient problem. When gradients vanish, earlier layers learn very slowly because the training signal fades.

Residual connections (or skip connections) solve this. Instead of learning an entirely new representation, the layer learns a residual update that is added to the original input.

$$Output = x + \text{Sublayer}(x)$$

Used around both the Masked Multi-Head Attention and FFN layers, residual connections help transformers train deeper networks, stabilize gradients, and preserve the original word information.

5. Layer Normalization

As data passes through many layers, activation values can grow too large or become too small, slowing down learning. Layer Normalization stabilizes these values by normalizing the features of each token independently, helping keep activations in a stable range during training.

For a token embedding:

$$x = [x_1, x_2, x_3]$$

LayerNorm computes:

The mean:

$$\mu = \frac{1}{n} \sum_{i=1}^n x_i$$

The variance:

$$\sigma^2 = \frac{1}{n} \sum_{i=1}^n (x_i - \mu)^2$$

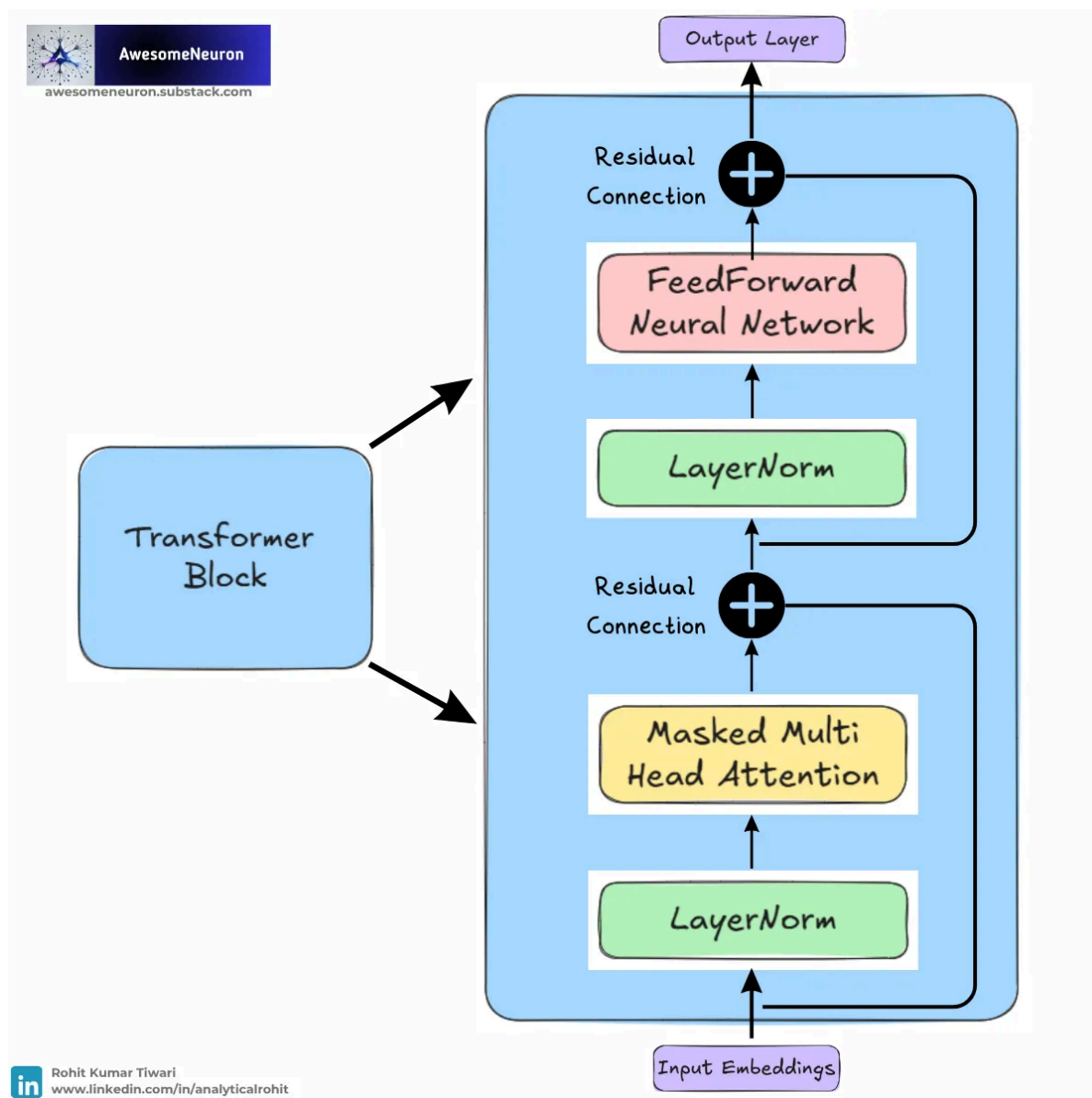
The normalized output, where epsilon is a tiny number added to prevent division by zero:

$$\hat{x}_i = \frac{x_i - \mu}{\sqrt{\sigma^2 + \epsilon}}$$

Layer normalization is applied multiple times inside each transformer block. This makes training significantly faster and more reliable.

The Transformer Block

A Transformer block brings together the main components of the Transformer architecture. GPT models are built by stacking many of these blocks, allowing token representations to become more informative at each layer. Modern GPT models use a pre-norm design, where layer normalization is applied before the attention and feedforward operations.



The flow through a Transformer block is:

1. **Layer Normalization**

Normalizes the input representations to improve training stability.

2. **Masked Multi-Head Attention**

Allows each token to gather information from itself and previous tokens while preventing access to future tokens.

3. **Residual Connection (Add)**

The original input is added back to the attention output, helping preserve information and improve gradient flow.

4. **Layer Normalization**

Re-normalizes the updated representations before further processing.

5. **Feedforward Network (FFN)**

Applies non linear transformations to learn more complex patterns and relationships.

6. **Residual Connection (Add)**

The input from before the second Layer Normalization is added to the FFN output, preserving information while incorporating the new transformations.

Note: Dropout is often applied after the attention and feedforward operations during training. This helps reduce overfitting and improves the model's ability to generalize.

Modern GPT models stack many transformer blocks on top of each other. Each block refines the token representations.

Here is the code showing Transformer Block in action.

```
1 class TransformerBlock(nn.Module):
2     def __init__(self, embed_dim, num_heads, hidden_dim):
3         super().__init__()
```

```

4
5     self.attention = MaskedMultiHeadAttention(embed_dim, num_heads)
6     self.norm1 = nn.LayerNorm(embed_dim)
7     self.ffn = FeedForward(embed_dim, hidden_dim)
8     self.norm2 = nn.LayerNorm(embed_dim)
9
10    def forward(self, x):
11
12        norm_x = self.norm1(x) # Layer normalization
13        attention_output = self.attention(norm_x) # Multi-head causal self attention
14        x = x + attention_output # Residual connections
15        norm_x = self.norm2(x) # Layer normalization
16        ffn_output = self.ffn(norm_x) # Feedforward neural network
17        x = x + ffn_output # Residual connections
18
19    return x

```

Output Layer

After passing through all the Transformer blocks, the model produces a context aware representation for each token. The model produces logits for every token position, but for next token generation it uses the logits from the final position in the sequence, which is “beginning” in this example.

This representation is then passed through the output layer:

1. Linear Layer (Unembedding)

The final vector is projected into a vector whose size matches the model’s vocabulary. The resulting values are called logits, with one logit for every possible token the model can generate.

2. Softmax Function

The logits are converted into a probability distribution over the entire vocabulary. Each value now represents the model’s estimated probability of a particular token being the next token in the sequence.

The model can then select the next token. For example, if the token "." has the highest probability, it may be chosen as the prediction.

Because large language models generate text one token at a time, the predicted token is appended to the original input.

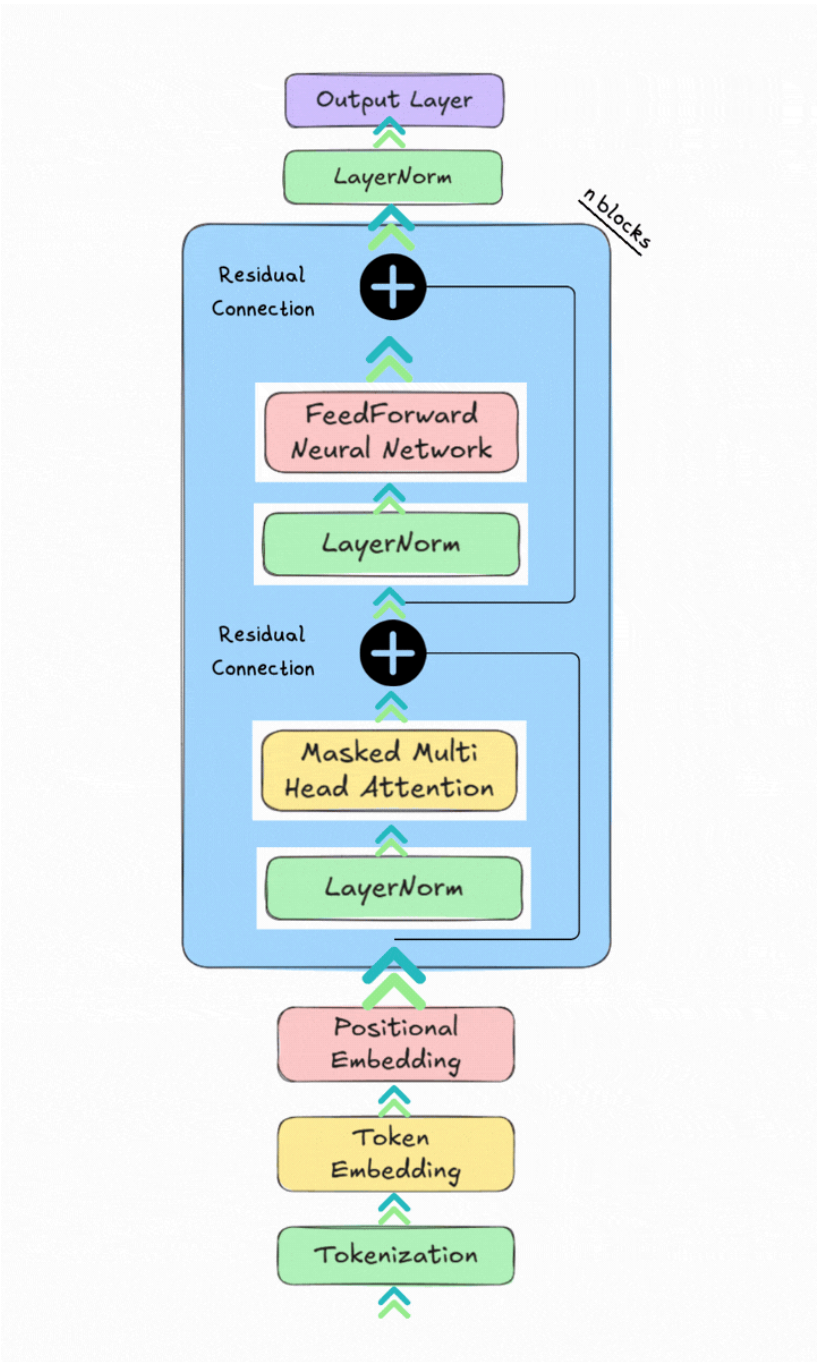
The sequence:

Every moment is a beginning

becomes:

Every moment is a beginning .

This updated sequence is fed back into the model, which repeats the same process, tokenization, token embeddings, positional embeddings, transformer blocks, and output layer, to predict the next token. By repeating this cycle, the model generates text incrementally, one token at a time.



Star and Clone the GitHub Repository!

Build a ChatGPT like LLM from scratch in PyTorch, explained step by step.

Code → <https://github.com/analyticalrohit/llms-from-scratch>

Missed a recent post? Catch up here 🙌

1. [7 Patterns for Smarter Multi-Agent Systems](#)
 2. [The Art and Science Behind Context Engineering](#)
 3. [Small Language Models are the Future of Agentic AI](#)
 4. [A Visual Guide to AI Agents](#)
 5. [A Visual Guide to Agentic RAG](#)
-

Liked this article? Make sure to ❤️ click the like button.

Know someone that would find this helpful? Make sure to 📄 share this post.

Get in touch

You can find me on [LinkedIn](#) | [GitHub](#) | [YouTube](#) | [X](#) | [Topmate](#)